

Class 4 Assignment: Building a Custom LLM

Choose data, train a tiny language model, test it on separate evals, and interact with it through a simple chat interface.

Overview

Use the ready-made notebook to understand the basics of a language model: corpus, tokens, vectors, embeddings, neural networks, and learning. It uses Andrej Karpathy's actual nanoGPT model, using PyTorch and a small word-token transformer. Choose your corpus, training steps, and learning rate, then submit one public GitHub repository URL with your executed notebook and evidence. This miniature model generates short sentences from a narrow corpus, not general chat answers.

What You Are Submitting

- Your executed `custom_llm.ipynb` for the starter-corpus experiment and the corpus-extension experiment, with settings, inspections, samples, plots, and eval results visible. Submit your own executed versions, not the unexecuted starter.
- Text samples from the untrained, halfway, and final model, using the same generation settings.
- A training/validation loss plot and the full table of measured values from the fixed evaluation panels.
- An inspection of one token's ID and embedding vector before and after training, next-token probabilities, and one real gradient and parameter update.
- A short explanation of the learning process, a temperature comparison, one limitation, and a proposed next experiment.
- The provided 48-case synthetic language eval suite, kept outside the training corpus, with complete untrained/final results for both the starter and expanded-corpus experiments.
- A working chat interface connected to your trained model, with source code, launch instructions, and evidence of at least 3 actual interactions. A terminal or notebook interface is enough.

How Evals Affect Your Assignment Grade

This assignment is graded out of 10 using the course framework: deliverable quality (4 points), testing & evaluation (3 points), and working result (3 points). The 48-case eval percentage is a model measurement, not your assignment grade. The runner does not calculate your grade.

- Deliverable quality (4 points): submit both executed experiments, readable source code, corpus sources and choices, and a clear README. Explain the model's learning process using your actual token, embedding, gradient, and loss evidence. Explain why you chose at least two extension categories and how your new teaching material addresses their gaps.
- Testing & evaluation (3 points): run all 48 unchanged cases before and after training in each experiment. That means four complete result sets: starter untrained, starter trained, expanded-corpus untrained, and expanded-corpus trained. Save every case, the CSV/JSON results, summaries, and separation checks. Compare all-case success, scorable accuracy, vocabulary coverage, group/category scores, and actual free continuations, alongside the existing loss evidence. Explain failures and whether changes reflect vocabulary coverage, learned patterns, or both.
- Working result (3 points): demonstrate your trained nanoGPT, rerunnable evals on your saved model, and a working interface that produces actual replies from that model. Include launch instructions, the model/run identity, and at least 3 real chat interactions with a screenshot or recording. A terminal or notebook interface is sufficient; a polished website is not required.

Running and interpreting the evals is required. Missing runs, omitted cases, a missing corpus-extension comparison, or unsupported conclusions reduce testing & evaluation credit. **A substantial penalty to the overall assignment grade will be considered if eval prompts, paired reference answers, answer keys, or eval outputs are included in the training corpus. The penalty may extend beyond the 3-point testing & evaluation category.** These results are not valid evaluation evidence. Remove the leaked material and rerun before submitting. Incomplete notebook/code or a nonworking model/interface also affects the relevant deliverable or working-result category. Partial credit follows the evidence provided; this is not an automatic all-or-nothing checklist.

There is no minimum model pass rate, leaderboard, or required numerical improvement. A low score, unknown-word cases, or an extension experiment that does not improve can still earn full testing & evaluation credit when the required experiments are complete, the method is valid, and the analysis explains what happened. Unknown-word cases count as zero in the model's all-case metric, not as an automatic deduction of the same percentage from your grade. A high score alone cannot replace valid evaluation, understanding, and a working deliverable.

Before submitting, make the README show a four-row comparison for the two experiments and their untrained/trained stages, link all four result sets, identify the extension categories and added data, discuss at least one concrete failure or limitation, and link your chat evidence.

Open the Notebook

- [Custom LLM sample project on GitHub](#)
- Local Jupyter or VS Code: install requirements.txt and open custom_llm.ipynb with that Python 3 environment. Colab generally includes PyTorch; notebook setup installs the PDF reader if absent. The pinned nanoGPT model source is provided.
- [Open custom_llm.ipynb in Google Colab. The default CPU runtime is enough.](#)

- Read [Karpathy's nanoGPT explanation](#) alongside the notebook. The model uses two blocks, four heads, 64-number embeddings, a 48-token context, PyTorch backpropagation, and AdamW. Whole-word tokenization is a classroom choice, not intrinsic to nanoGPT. Classroom additions make data, evaluation, and learning visible.

Expand Your Corpus with Files

- Put PDF, TXT or Markdown files in corpus/ beside the notebook, such as corpus/report.pdf, corpus/notes.txt or corpus/research/summary.md. Subfolders work. You do not need to paste the text into the code.
- In Colab, run sections 1 and 2 once to create /content/corpus. Refresh the left Files sidebar and upload your files into that folder. Opening a GitHub notebook does not copy its folders or your local files into Colab. Save your source files elsewhere too; runtime storage is temporary.
- Keep CORPUS = "classroom" to add your files to the teaching sentences, or set CORPUS = "folder" to use only your files. Folder-only mode requires at least 100 distinct extracted passages. CORPUS_FOLDER selects the folder; its default is "corpus".
- Select Run All from the top. Section 3 shows the imported files, text previews, passage counts and warnings. After training, load the new ZIP's checkpoint.json in the embedding viewer. Adding files does not instantly update the model: the network must train on their text.
- PDFs need extractable text; run OCR on scans first. Unreadable, encrypted or entirely textless files stop with an error naming the file. Partly textless PDFs produce page warnings. Inspect the extracted text for missing pages or confusing reading order. TXT and MD must use UTF-8; Markdown is plain text, and links/code are not fetched or executed.
- Long documents are split automatically into non-overlapping passages of at most 47 word/punctuation tokens, keeping sentence or line boundaries when possible. A passage is the short training example the notebook calls a document. The model keeps the 509 most frequent training token types; other types become UNK. Inspect both unknown-token rates before assuming a larger corpus adds useful information.
- Review corpus_manifest.json for file sources, previews, warnings and duplicate counts, and vocabulary_report.json for vocabulary coverage. Limits are 50 supported files, 25 MB each, 100 MB total, 200 pages per PDF and 2 million extracted characters per file. Hidden files, symbolic links, unsupported formats and corpus/README.md are ignored.
- Use only material you have permission to use and share. Added files in corpus/ are Git-ignored, but the results ZIP contains extracted text, filenames/hashes and model weights; the executed notebook also exposes examples. Review all artifacts before publishing. Git-ignore is not a privacy guarantee for derived results.

Your Three Choices

- Corpus: first run the supplied classroom corpus, then expand it with focused files for at least two extension-eval categories. Explain your sources, the unique passages added, and what patterns they could teach. Folder-only mode remains available for further experiments.
- Training steps: a positive whole number of weight updates. Try 10 for setup, then 3,000 as a starting training budget. A step is not an entire pass through the corpus. Runtime and sample quality depend on your machine, data, and choices.
- Learning rate: the initial size of the optimizer's updates. Start with 0.001. The notebook uses warmup and cosine decay during training. Explain why excessively large or small updates could be a problem.
- Edit those three settings in section 1 and write your prediction before training. You can keep the defaults, provided you explain your choices.
- The 48-token context window keeps this model small enough to inspect. Longer text is split into passages, not silently truncated. Use data you have permission to share, without confidential or personal records.
- Build the vocabulary only from training text and report both training and held-out unknown-token rates. Other settings are optional experiments. Change one at a time and explain it. Complete and understand the baseline notebook first.

Evaluate the Model Fairly

- Keep the same split, evaluation panels, seed, and baseline generation settings before and after training. Duplicate passages are removed before a 90/10 passage split; validation passages never supply weight updates. Passages from the same source file can appear in both sets, so this does not test generalization to unseen source files.
- The loss plot uses fixed panels of at most 20 training and 20 validation documents, averaging non-padding next-token targets. Report every measured value and both panel sizes. These are small estimates, not full-corpus measurements.
- Show all saved samples, including empty or garbled strings. Use the same starting token and sampling seed for the temperature comparison. Different corpora and vocabularies do not produce directly comparable loss scores.
- Falling training loss alone does not demonstrate generalization. Compare held-out loss and samples too. The synthetic corpus deliberately repeats contexts. Held-out sentences share templates with training, so plausible output does not demonstrate broad knowledge or generalization to new templates. Report lack of improvement honestly.

Run the Provided Language Evals

An eval is a fixed test: a prompt, a rule for judging the response, and the model's actual result. The corpus is study material; the eval suite is the exam. Training on the exam can make memorization look like learning.

- Use the 48 synthetic language tests already provided in `evals/language_evals.json`: 16 reserved prompts for starter-corpus patterns, 8 new phrasings using familiar words, and 24 extension challenges. The extension skills are grammar, opposites, negation, references, sequence, spatial relations, everyday knowledge, and categories/analogies. You do not need to invent the eval suite.
- Read all 48 language eval cases and the runnable eval guide in the sample repository. Keep the cases, four answer choices, answer key, and scoring method unchanged across experiments. Each case includes its category and an explanation. The suite is stored in `evals/`, outside `corpus/`.
- Keep eval prompts, reference answers, scoring rules, and generated eval outputs out of all training inputs, including PDFs, Markdown, generated classroom sentences, and copied chat logs. Do not use eval text to build the vocabulary or update weights. Ordinary words and underlying subject knowledge may overlap; the test items themselves must stay separate. Keep `CORPUS_FOLDER` pointed only at `corpus/`, never at the whole project. **Including eval material in the training corpus may result in a substantial penalty to your overall assignment grade, as described in the grading section.**
- The notebook removes generated classroom sentences containing reserved test prefixes before splitting data or building the vocabulary. It rejects exact test prefixes in imported files and rejects corpus folders that include `evals/` or the project root. Inspect `eval_separation.json`, the file manifest, and your training text. These checks do not catch all paraphrases or copied answer lists. The existing 90/10 loss panels remain separate measurements.
- Run All in the updated notebook: section 6b evaluates the untrained model; section 8b repeats the same 48 tests after training. The runner sends only the prefix to your nanoGPT, without answer choices or the answer key, and never updates weights. It saves every result and a free text continuation. You can also rerun saved weights with `run_evals.py`; the guide provides the exact command.
- The score is 1 when the model assigns the highest probability to the correct word among four choices, and 0 otherwise. This scores next-word selection, not the separately saved free continuation. Ties receive no credit. Cases with unknown prompt/answer words or excessive context are marked unscorable and receive zero in the all-case success rate. Report all-case success, scorable accuracy, coverage, and group/category results. Never drop failures. There is no minimum pass rate.
- Save your starter-corpus experiment, choose at least two extension categories, and add varied teaching examples with different wording, people, or situations to `corpus/`. Train a fresh model and rerun all 48 fixed tests. Compare scores, vocabulary coverage, and actual continuations. More training on the original corpus cannot supply missing vocabulary; new data may help but does not guarantee success. Since these public tests guide improvement, call this a development benchmark. A claim about unseen generalization would require additional tests that never guided your choices.

Example: Corpus vs. Evals

Two examples from the provided suite:

Supported pattern: the training corpus includes other sentences connecting surgeon, patient, care, and hospital. One reserved test prefix is "the report about the surgeon explains the". Its choices are patient, delivery, fruit, and traffic, with patient as the intended answer. All generated sentences containing that exact test prefix are withheld.

Extension example: "ava did not buy tea . she bought milk . ava bought" should favor milk over tea, bread, or rice. This requires vocabulary and a negation/correction pattern absent from the starter. Teach those ideas using different examples; do not put this test story, its answer list, or its outputs in the corpus.

Flow: corpus/ → training → model weights. Separately: eval prompt → trained model → response → compare with the saved rule. The expected answer never enters the model's prompt or its weight updates.

For the surgeon example, patient as the highest-probability choice earns 1 and traffic earns 0. The model also generates an unrestricted continuation, which may differ from its multiple-choice selection. Save and inspect both. A correct selection shows a narrow learned pattern, not general language understanding.

Repository layout: corpus/ holds teaching sources; evals/language_evals.json holds the fixed tests; run_evals.py runs inference and scoring; llm_runs/ stores results and model weights; chat.py provides the terminal interface. The notebook integrates the runner and a prompt/reply interface. Keep test files and result folders outside every training input.

Chat with Your Trained Model

- Provide a working interface where a user can enter text, see a response from your trained nanoGPT, and enter another prompt. A terminal loop, a Colab/Jupyter input loop, or an HTML/web interface is enough. Include the interface code and exact launch instructions. A hosted website is optional.
- Use the trained model from your experiment. A separate app should load model.pt and its saved vocabulary; checkpoint.json contains embedding-viewer data and is not the full inference model. A notebook interface may use the trained model already in memory. A web page must actually run or connect to the model. Do not substitute canned answers or another model's API.
- Label the interface as a tiny language model. It may continue a sentence rather than answer a question. Show or explain unknown words, the 48-token context limit, and whether each prompt starts fresh. Conversation memory is optional. Generating replies must not retrain the model or add chat messages to the corpus.
- Use the provided notebook section 10 or chat.py, or build your own interface. Include a screenshot or short recording and a saved transcript with at least 3 actual prompts and responses, including a failure or limitation. Identify the trained run used. The embedding viewer alone is not a chat interface.

Save Your Results

Each Run All creates a new `llm_runs/` folder and a ZIP. In Colab, download the ZIP before ending the session. Also download the executed notebook separately after the run; the results ZIP does not contain the currently open notebook.

- The folder contains `config.json`, `corpus.txt`, `corpus_manifest.json`, `vocabulary_report.json`, `split.json`, `tokenization.json`, `inspection.json`, `history.json`, `training.csv`, `training_summary.json`, `checkpoint.json`, `model.pt`, `training_curves.svg`, the `samples/` timeline, and `temperature_comparison.json`.
- If you interrupt training, continue through the inspection, plot, and download cells. Report the completed steps and interruption. Other errors require fixing the cause and rerunning from the top; a complete results ZIP is not guaranteed after an error.
- The updated results ZIP includes the fixed eval cases, untrained/final CSV and JSON results, summaries, `eval_separation.json`, and `model_untrained.pt`. The notebook chat cell saves `chat_transcript.json` and refreshes the ZIP. Submit the runnable notebook and helper scripts as source code alongside the results.

README Requirements

The README is the grading entry point. A reader should be able to follow your experiment, inspect the evidence, and understand your explanation without rerunning the notebook.

- A brief overview, the sources and permissions for your corpus files, and instructions to open and run your notebook. Explain how you checked PDF extraction and any warnings.
- Your three choices and reasons, plus the number of unique passages, vocabulary size, training/held-out unknown-token rates, and the train/validation split. Link `corpus_manifest.json` and `vocabulary_report.json` when sharing is permitted.
- What you expected before training, followed by what you actually observed in the same run.
- Actual completed steps, elapsed time, hardware, and the model's parameter count. Identify interrupted or failed runs clearly.
- Explain corpus, tokens, IDs, vectors, embeddings, neural-network weights, loss, and learning using actual notebook examples. Trace one word from text to its ID and 64-number vector, then explain one saved gradient and weight update.
- Explain how attention uses earlier context, how probabilities become generated word tokens, and how temperature changes sampling without updating weights. Finish with one observed limitation and one proposed next experiment.
- Explain how to rerun your eval suite and launch your chat interface, including dependencies and how to obtain/load your saved model. Describe the eval scoring rules, before/after results, leakage checks, and one observed chat limitation.

Suggested Workflow

1. Open the sample notebook, save your own copy, and read the explanatory cells as you go.
2. Choose your corpus, training steps, and learning rate. Write your reasons and prediction. A 10-step run checks setup; use a meaningful training budget for the final experiment.
3. Keep the provided 48-case suite fixed. Run the notebook from the top so the separate evals execute before and after training. Keep the starter run and its outputs. Then add different teaching examples for at least two extension categories and run a second experiment.
4. Download `embedding-viewer.html` from the sample repository, open it locally, and load your `checkpoint.json`. Compare a word's initial/final 64D vector and neighbors, inspect the first update, and read the sample timeline alongside both loss curves. PCA compresses the map; cosine neighbors use the full vector space. Compare the three temperatures without retraining.
 - Compare all 48 cases across both experiments, including unknown-word coverage and failures. Try the supplied chat interface or your own, save at least 3 actual interactions, and verify the launch instructions.
5. Save the results ZIP and the executed notebook separately, write your explanation in the README, and publish your notebook and selected evidence on GitHub.

Starter Prompt for Your AI Assistant

Help me work through `custom_llm.ipynb` for Class 4. Before training, ask for my training steps and learning rate, explain the choices, and help me write a prediction. Start with the classroom corpus and use the supplied 48-case eval suite unchanged. Keep eval prompts, answer keys, and outputs out of training and vocabulary building. Run the notebook's before/after evals and explain the difference between four-choice scores, free continuations, and vocabulary coverage. Help me choose at least two extension categories, write different teaching examples in corpus/, and train a second model. Compare the actual results without hiding failures. Keep the existing loss panels and inspections; ask me to explain tokens, embeddings, probabilities, gradients, and weight changes in my own words. Help me use the notebook or terminal chat interface with my trained model, save at least 3 real interactions, and write an honest README. Do not invent results, copy the tests into training data, or substitute another model.

Evidence Required in the README

- Show the untrained, halfway, and final text samples, linking the full saved files. Explain at least one visible change or lack of change.
- Embed `training_curves.svg` and include the full loss table from `history.json`. State that these are fixed training and validation panels, each with at most 20 documents.

- Link tokenization.json and inspection.json. Include one word-to-ID-to-vector example, the vector before/after, the first parameter's value/gradient/update, and one next-token probability comparison. Explain what each means.
- Link your executed notebook, config.json, training.csv, training_summary.json, and temperature_comparison.json. Explain which data and settings stayed fixed, what changed in training, and what changed only at inference.
- Link the unchanged 48-case suite, runner, and complete untrained/final results for both experiments. Report scores, category breakdowns, vocabulary coverage, actual continuations, and corpus-separation checks. Explain your added teaching material and any improvements or failures. These are public development tests, not an unseen final benchmark.
- Link the chat-interface code and launch instructions, and show a screenshot or recording plus a transcript of at least 3 actual prompts and replies. Identify the model/run used.

Submission Checklist

1. After the final run, save your notebook with all outputs. Upload or push that .ipynb file, your README, and selected results to your public GitHub repository. In Colab, use File → Download → Download .ipynb. Do not clear the outputs. Open the notebook on GitHub and confirm that the final inspections, losses, samples, and plot are visible.
 - Include the fixed eval suite, runner, complete results for both experiments, and chat-interface code. Test the documented run and launch instructions with your saved model. Keep the source code in your repository and save both result ZIPs and executed notebooks.
2. Open your repository signed out and verify that the notebook, plot, sample files, and evidence links are accessible.
3. Keep the complete results ZIP locally. checkpoint.json stores initial/final embeddings for the viewer; model.pt stores the full network for inference. Neither is an exact training-resume file. Include the corpus or a reproducible source link when sharing is permitted.
4. [Submit your GitHub repository](#)

Learning Focus

The central purpose is understanding how data becomes predictions and how a neural network learns. Your explanation, choices, and visible evidence should support one another. A bigger network, longer training run, or more convincing sample does not replace an explanation. Report limitations rather than claiming the model understands language from a few plausible strings. Losses across different corpora are not a class ranking.

Definition of Done

- The notebook completes with your chosen corpus and settings, and your README records the actual training budget.
- You compare the untrained, halfway, and final model using fixed evaluation settings and show all measured losses and saved samples.
- You can point to an actual token ID, embedding vector, next-token probability, gradient, and weight update, and explain how they connect.
- You can explain the role of the corpus, the neural network, held-out data, attention/context, and temperature, plus one limitation.
- All 48 provided evals run before and after training in both experiments. You save every result, extend the corpus for at least two challenge categories, and explain how eval material stayed out of training.
- Your chat interface accepts new prompts and generates responses from your trained model; the README shows how to launch it and includes actual interaction evidence.

Single Deliverable

- One public GitHub repository URL through the course submission portal in bcourses. Its README contains your explanation and evidence, with the executed training notebook, separate eval cases and runner, all eval results, and working chat-interface code and launch instructions.

Scope

- Use the supplied nanoGPT model. Writing the network yourself is optional. PyTorch runs the small model on CPU without an API key or pretrained weights. The vocabulary uses words and punctuation, not characters.
- A working chat interface is required; a terminal or notebook loop is enough. A website, backend service, deployment, GPU purchase, or large-model training is optional. Keep the experiment small enough to inspect.
- [Karpathy's nanoGPT source](#) is the core reference. The larger [GPT video project](#) is an optional explanation of GPT training. The current lab already uses PyTorch and word tokens.
- Two experiments are required: the starter corpus and your corpus extension for at least two eval categories. More training steps or further experiments are optional. Explain outcomes honestly; improvement is not guaranteed.